ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CO5085 – Học sâu và ứng dụng trong thị giác máy tính

BÁO CÁO BÀI TẬP E1–E3

Phân loại ảnh Fashion-MNIST
Mạng tuyến tính, CNN, attention và mạng hồi tiếp

Học kỳ 261 – Năm học 2026–2027

Giảng viên: TS. Lê Thành Sách
TS. Nguyễn Quốc Minh
Sinh viên 1: Phạm Duy Anh - 2310139
Sinh viên 2: Nguyễn Phan Tuấn Duy - 2310491

TP. Hồ Chí Minh, Tháng 11 năm 2026

Mục lục

1 Mục tiêu và phạm vi bằng chứng **2**

2 Dữ liệu, EDA và protocol chung **3**
2.1 Fashion-MNIST và cách chia dữ liệu . . . 3
2.2 EDA và quyết định tiền xử lý . . . 3

3 Môi trường và vòng lặp thực nghiệm **4**
3.1 Môi trường đã ghi nhận . . . 4
3.2 Huấn luyện và chọn checkpoint . . . 4
3.3 Cách đọc số liệu và giới hạn log . . . 4

4 E1: Softmax, MLP và CNN **5**
4.1 Giả thuyết và kiến trúc . . . 5
4.2 Kết quả định lượng và chi phí . . . 6
4.3 Learning curves và overfitting . . . 6
4.4 Ma trận nhầm lẫn và mẫu lỗi . . . 7

5 E2: Multi-head self-attention và tokenization **9**
5.1 Từ token tới attention . . . 9
5.2 Encoder phân loại ảnh . . . 9
5.3 Kiểm chứng tương đương với PyTorch . . . 10
5.4 Ba tokenizer và sáu cấu hình . . . 11
5.5 Động học học và lỗi E2 . . . 12

6 E3: LSTM và GRU trên chuỗi ảnh **13**
6.1 Ánh xạ ảnh thành chuỗi . . . 13
6.2 Cơ chế cổng . . . 13
6.3 Kết quả E3 và đối chiếu baseline . . . 14
6.4 Learning curves và lỗi E3 . . . 15

7 Tổng hợp, hạn chế và hướng phát triển **16**
7.1 Trade-off quan sát được . . . 16
7.2 Giới hạn bằng chứng . . . 16
7.3 Hướng cải thiện có thể kiểm chứng . . . 16

8 Tái lập và cấu trúc tài liệu **17**
8.1 Notebook, artifact và báo cáo . . . 17
8.2 Định danh và cách kiểm toán . . . 17

A Đường cong và ma trận bổ sung **18**

1 Mục tiêu và phạm vi bằng chứng

Báo cáo khảo sát cách biểu diễn ảnh ảnh hưởng tới chất lượng và chi phí phân loại. E1 đi từ softmax tuyến tính tới MLP và CNN; E2 tự cài đặt multi-head self-attention (MSA), đối chiếu với PyTorch và thay tokenizer; E3 chuyển ảnh thành chuỗi để so sánh LSTM và GRU. Cùng một split Fashion-MNIST giúp việc đối chiếu không bị thay đổi tập đánh giá chi phối.

Các bảng được sinh từ `artifacts/E1`, `E2`, `E3`, nhánh `full`, không sử dụng cột preview trên website. Có 3 cấu hình E1, 6 cấu hình E2 và 4 cấu hình E3. Mỗi cấu hình có ngân sách 20 epoch, seed 42. Checkpoint dùng để đánh giá được chọn theo validation accuracy, hòa thì lấy validation loss thấp hơn; test không dùng cho việc chọn checkpoint.

Bảng 1: Truy vết yêu cầu trong `e1_e3_co5085.md`.

Bài	Yêu cầu và bằng chứng	Phần
Chung	Dataset, split, môi trường, phép so sánh có kiểm soát	2, 3
E1	Softmax, MLP, CNN; vòng lặp PyTorch, accuracy và chi phí	4
E2	MSA tự viết và thư viện; kiểm chứng output/gradient; tokenizer	5
E3	Ảnh thành chuỗi; LSTM/GRU; so với MLP/CNN	6
Chung	Hình, sai lầm, hạn chế và tái lập	7, 8
Nộp bài	Video chưa được cung cấp	Khai báo dưới đây

Kết quả nổi bật: CNN E1 đạt 91,95% test accuracy với 50.186 tham số; trong E2, scratch CNN-stem đạt 90,20%; trong E3, GRU-row đạt 89,87%. Những kết quả này chỉ mô tả study hiện tại, không chứng minh CNN luôn hơn Transformer hoặc RNN trên mọi bài toán.

Khai báo sử dụng AI

AI được sử dụng để hỗ trợ thiết kế giao diện website và trực quan hóa kết quả; nghiên cứu tài liệu, đối chiếu nguồn tham khảo; xây dựng, rà soát và debug mã nguồn; thiết kế cơ chế điều phối huấn luyện song song bằng hai tiến trình trên hai GPU; tổ chức artifact, kiểm tra tính nhất quán và soạn thảo báo cáo LaTeX. Các số đo trong báo cáo được lấy từ artifact và notebook đã chạy, không thay bằng kết quả do AI tự tạo. Hỗ trợ về mã điều phối không đồng nghĩa AI đã thực hiện thêm các lượt huấn luyện mới trong quá trình biên soạn báo cáo. Hai thành viên chịu trách nhiệm về nội dung và kết quả được trình bày; cần tự rà soát nguồn, kiểm tra mã, hiểu phương pháp và có khả năng giải thích, bảo vệ toàn bộ nội dung trước khi nộp. Việc dùng AI không thay thế các yêu cầu tự thực hiện và hiểu bài trong đề. URL video chưa được cung cấp và không được tự bổ sung.

2 Dữ liệu, EDA và protocol chung

2.1 Fashion-MNIST và cách chia dữ liệu

Fashion-MNIST gồm ảnh mức xám 28×28 thuộc 10 loại sản phẩm, có official train 60.000 ảnh và test 10.000 ảnh [4, 5]. Thí nghiệm dùng `torchvision.datasets.FashionMNIST`; official train được chia phân tầng thành 54.000 train và 6.000 validation bằng seed 42. Mỗi lớp có 5.400/600/1.000 ảnh theo ba split. Chỉ số split được giữ ổn định giữa các bài.

SHA-256 split được xác nhận giống nhau trong config của cả 13 cấu hình:

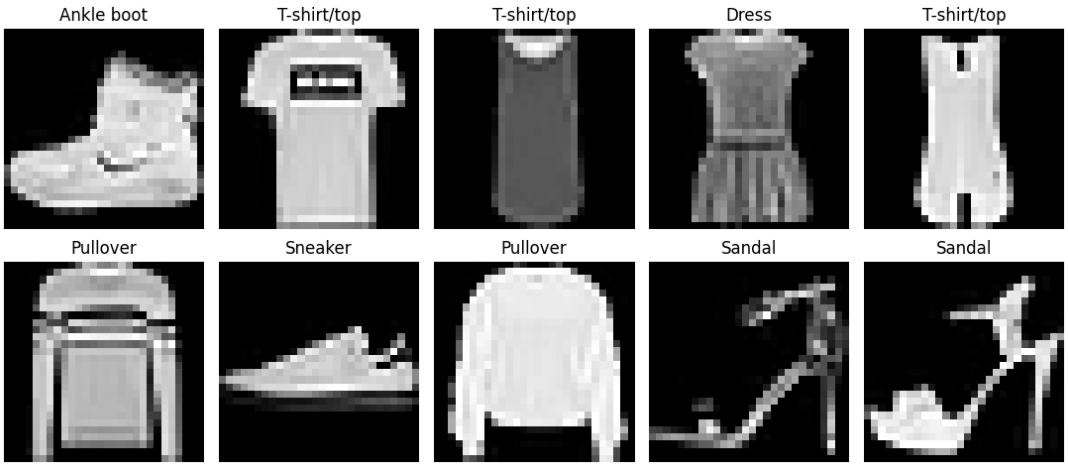
```
bbadd2fc92455ce7f7ca22890516a1f02c47a663679ec246709d7cf0cfdcb075
```

Không thể kết luận mọi khác biệt đều do kiến trúc, nhưng việc giữ split loại bỏ một nguồn nhiễu rõ ràng. Các tham số optimizer vẫn khác theo bài, nên so sánh liên bài là so sánh các hệ thống được triển khai với recipe tương ứng.

2.2 EDA và quyết định tiền xử lý

Các lớp cân bằng nên không sử dụng weighted sampler hoặc class weights. Ảnh là một kênh, hình dạng thống nhất; không cần resize để dùng với các mô hình nhỏ. Hình 1 cho thấy khác biệt đường viền hữu ích giữa quần, túi và giày, nhưng áo sơ mi, áo phông, áo len hoặc áo khoác có thể gần nhau về hình dạng khi mất thông tin màu và texture.

Đầu vào được đổi về $[0, 1]$ rồi chuẩn hóa bằng $\mu = 0{,}2860405743$ và $\sigma = 0{,}3530242443$. Theo code thực tế, hai số này tính trên *toàn bộ official train 60.000 ảnh trước khi tách validation*, không phải chỉ trên 54.000 ảnh train. Test không tham gia tính thống kê. Đây là mức dùng thông tin phân bố validation trong preprocessing; báo cáo không gọi pipeline này là hoàn toàn train-only. Một lần chạy cải thiện sau này nên tính lại thống kê chỉ từ train và tạo study mới thay vì sửa số đo hiện có.



Hình 1: Mẫu ảnh trong output EDA của notebook E1 đã chạy. Grid minh họa không thay thế thống kê số lượng lớp.

3 Môi trường và vòng lặp thực nghiệm

3.1 Môi trường đã ghi nhận

Các environment record dùng Python 3.13.16, PyTorch 2.11.0+cu130, torchvision 0.26.0+cu130, scikit-learn 1.6.1, CUDA13.0 và Tesla T4. Các phiên Colab có kernel Linux khác nhau; thời gian được xem là đo thực nghiệm ở phiên tương ứng, không coi là benchmark GPU cô lập hoàn toàn.

E1 dùng Adam, LR 10^{-3}. E2 dùng AdamW, LR 10^{-3}, weight decay 10^{-4}. E3 dùng Adam, LR 10^{-3} và clip gradient norm 1. Tất cả dùng batch 128, cross entropy, 20 epoch và không augmentation. Chuẩn hóa và tập đánh giá là chung. Softmax trong loss được xử lý nội bộ bởi cross entropy; model trả logits, không gọi softmax trước loss.

3.2 Huấn luyện và chọn checkpoint

Snippet dưới rút gọn vòng lặp PyTorch trong notebook: cập nhật model ở train, tính validation với `eval()` và `no_grad()`, lưu best sau khi so sánh. E3 thêm gradient clipping ngay trước optimizer step.

```
model.train()
for images, labels in train_loader:
    optimizer.zero_grad(set_to_none=True)
    logits = model(images.to(device))
    loss = F.cross_entropy(logits, labels.to(device))
    loss.backward()
    # E3 only: clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()

model.eval()
with torch.no_grad():
    for images, labels in val_loader:
        logits = model(images.to(device))
        # Sum loss by number of images; count correct labels.
# Save best when val_accuracy rises, or loss breaks a tie.
```

Loss toàn tập là tổng loss theo số ảnh chia tổng ảnh, không phải trung bình không trọng số của các batch. Accuracy cũng đếm trên toàn tập. Một batch cuối nhỏ không được tính ngang trọng số với batch đầy.

3.3 Cách đọc số liệu và giới hạn log

Trong artifact, epoch bắt đầu từ 0; bảng báo cáo cộng 1 để hiển thị thứ tự epoch 1–20. Peak memory và kích thước weights được chia cho 2^{20} dù tên trường có đuôi MB; ở đây gọi MiB. Latency là mili giây mỗi ảnh khi batch 32, không phải độ trễ một yêu cầu batch 1. E1–E3 không ghi macro-F1 vào metrics, vì vậy không suy diễn F 1 từ accuracy. E2 chỉ có đường cong PNG trong bundle; không tự dựng lại history CSV từ hình. E1/E3 có history ở một số bản export cục bộ, còn bảng chính dùng metrics đã xuất.

4 E1: Softmax, MLP và CNN

4.1 Giả thuyết và kiến trúc

Với $x \in \mathbb{R}^{784}$, mô hình tuyến tính cho logits $z = Wx + b$ và xác suất lớp $p_k = \exp(z_k)/\sum_j \exp(z_j)$. Cross-entropy với nhãn y là $-\log p_y$. Đây là baseline để xem mức phân biệt đạt được từ phép kết hợp tuyến tính các pixel, không có tương tác phi tuyến hay chia sẻ tham số theo không gian.

MLP dùng hai tầng ẩn 256 và 128 với ReLU. Nó học tương tác phi tuyến nhưng flatten ảnh làm mất cấu trúc lân cận rõ ràng: các pixel có vị trí riêng trong ma trận trọng số. CNN dùng kernel 3×3 và pooling để xây dựng đặc trưng cục bộ, chia sẻ trọng số qua vị trí.

Bảng 2: Kiến trúc E1 và hình dạng tensor (bỏ chiều batch).

Model	Pipeline	Tham số
Softmax	Flatten $1 \times 28 \times 28 \to 784$; Linear $784 \to 10$	7.850
MLP	Flatten; Linear $784 \to 256 \to 128 \to 10$; ReLU ở tầng ẩn	235.146
CNN	Conv $1 \to 32$, ReLU, Pool 2; Conv $32 \to 64$, ReLU, Pool 2; Linear $64 \cdot 7 \cdot 7 \to 10$	50.186

```
self.features = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2))
self.head = nn.Linear(64 * 7 * 7, 10)
# forward:
return self.head(self.features(images).flatten(1))
```

Các kernel không thay đổi kích thước nhờ padding 1; hai lần pooling giảm $28 \to 14 \to 7$. Bộ phân loại cuối nhận 3.136 đặc trưng. CNN ít tham số hơn MLP do dùng lại kernel, dù tính toán convolution và activation trung gian có thể tốn bộ nhớ hơn.

Nguồn: `E1/e1_fashion_mnist_baselines.ipynb`.

4.2 Kết quả định lượng và chi phí

Bảng 3: E1: kết quả đo; best epoch đổi từ chỉ số 0 sang thứ tự 1–20.

ID	Val (%)	Test (%)	Test loss	Best*	Tham số
softmax	86.50	84.10	0.450	16	7,850
mlp	90.58	89.33	0.370	16	235,146
cnn	92.98	91.95	0.253	13	50,186

Bảng 4: E1: chi phí đo trên Tesla T4. Ký hiệu MB trong JSON được tính theo 2^{20} byte.

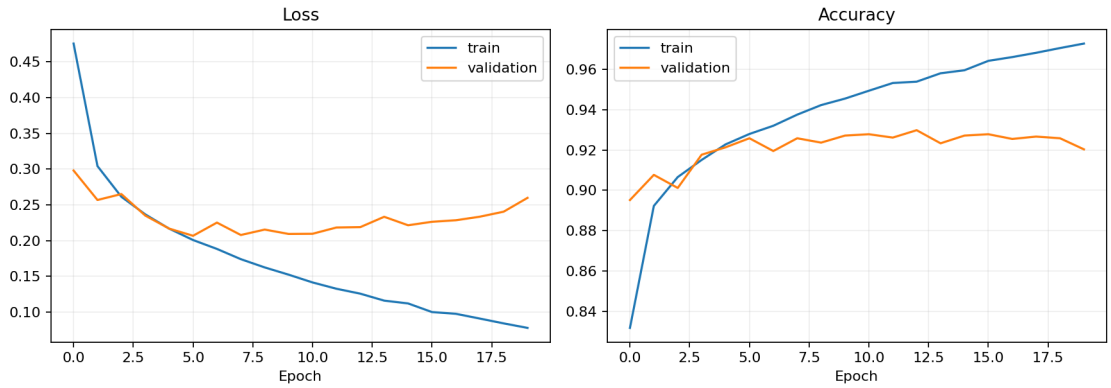
ID	VRAM train (MiB)	Weights (MiB)	ms/ảnh (B32)
softmax	18.04	0.030	0.0055
mlp	23.67	0.897	0.0045
cnn	42.92	0.191	0.0196

CNN hơn softmax 7,85 điểm phần trăm test accuracy và hơn MLP 2,62 điểm. Chỉ tiêu validation cũng xếp CNN đầu trong ba cấu hình, nên lựa chọn CNN không dựa trên test. MLP cải thiện so với tuyến tính nhưng dùng khoảng 4,69 lần tham số của CNN. Điều đó cho thấy số tham số đơn lẻ không dự báo chất lượng: cách chia sẻ và bố trí tham số phù hợp ảnh có vai trò lớn trong study này.

CNN có latency B 32 lớn hơn MLP dù ít tham số. Độ trễ phụ thuộc loại phép toán, kích thước activation và hiệu quả kernel, không chỉ dung lượng weights. Softmax nhỏ nhất nhưng đánh đổi đáng kể độ chính xác. Với ứng dụng ưu tiên footprint hơn chất lượng, baseline tuyến tính vẫn là điểm tham chiếu hữu ích.

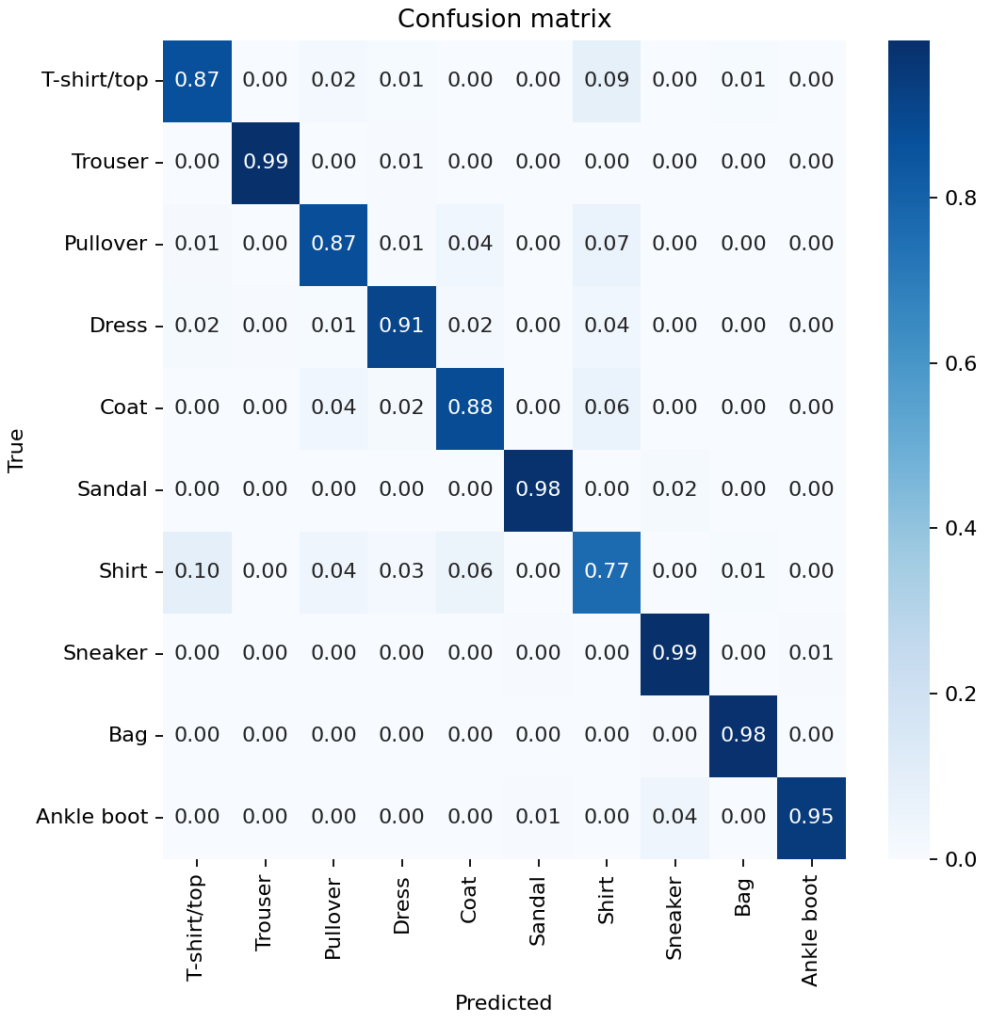
4.3 Learning curves và overfitting

Hình 2 cho phép nhìn train/validation theo epoch thay vì chỉ số cuối. Best CNN ở epoch 13, trước ngân sách 20: tiếp tục tối ưu train không bảo đảm tốt hơn trên validation. Không nên kết luận mô hình đã hội tụ hoàn toàn chỉ vì đã chạy đủ số epoch. Việc dùng best checkpoint giảm phụ thuộc vào epoch cuối nhưng không sửa các hạn chế một seed hoặc recipe chung.



Hình 2: Learning curves E1 CNN, từ export `E1/full/cnn`. Các mốc trên hình dùng chỉ số epoch của notebook.

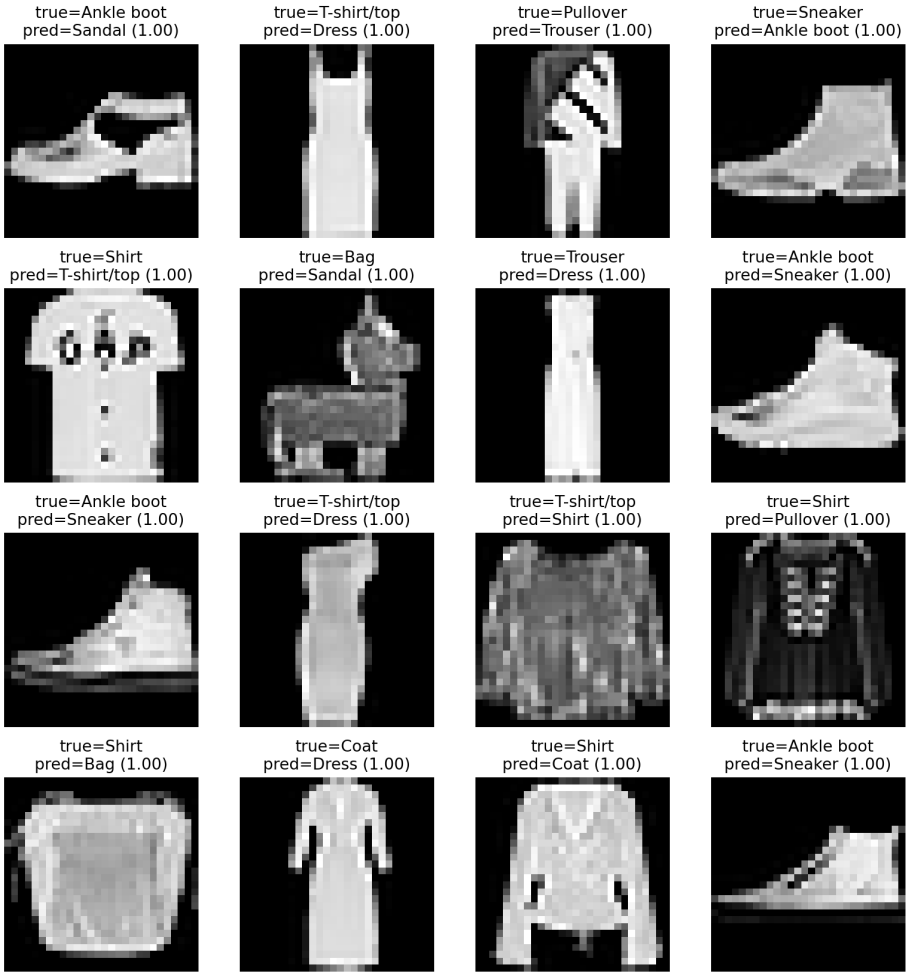
4.4 Ma trận nhầm lẫn và mẫu lỗi



Hình 3: Ma trận nhầm lẫn E1 CNN đã chuẩn hóa theo nhãn thật; không phải số đếm tuyệt đối.

Nhóm Shirt khó hơn các lớp Trouser, Bag và Sneaker trong ma trận này. Shirt có thể bị nhầm thành T-shirt/top, Pullover hoặc Coat; các lớp này chia sẻ dáng thân áo và tay áo. Ngược lại, đường viền dài tách biệt của Trouser là tín hiệu đơn giản hơn. Kết luận gắn với hình đã lưu, không phải đánh giá lại bằng dữ liệu mới.

Các giá trị trên hình làm tròn hai chữ số; không dùng chúng để tính lại macro-F1 chính xác. Hình chuẩn hóa giúp thấy lớp yếu, nhưng cần counts/predictions gốc nếu muốn tái tính mọi metric đến nhiều chữ số thập phân.



Hình 4: Mẫu lỗi CNN E1 đã lưu trong `failures.png`; nhãn thật và dự đoán in phía trên ảnh.

Một số mẫu giày bị nhầm giữa Sneaker và Ankle boot; ảnh áo bị nhầm giữa Shirt, Coat và Pullover. Đây là các trường hợp cần giữ trong phân tích thay vì chỉ trình bày ảnh dự đoán đúng. Mức confidence hiển thị gần 1 ở một số lỗi cho thấy điểm softmax cao không đồng nghĩa dự đoán đáng tin. Chưa có đánh giá calibration/ECE nên không khẳng định xác suất đã hiệu chỉnh.

Hướng cải thiện gồm augmentation nhẹ có kiểm soát, regularization hoặc kiến trúc tốt hơn. Cần chạy đối chứng trên validation trước khi khẳng định cải thiện; các đề xuất không được cộng vào thành tích E1 hiện tại.

5 E2: Multi-head self-attention và tokenization

5.1 Từ token tới attention

Cho $X \in \mathbb{R}^{B\times N\times d}$, mỗi head có $d_h = d/H$. Với các phép chiếu học được, attention [3] được tính như sau:

$$Q_h = XW_h^Q + b_h^Q, \quad K_h = XW_h^K + b_h^K, \quad V_h = XW_h^V + b_h^V, \tag{1}$$

$$A_h = \text{softmax}\left(Q_h K_h^\top / \sqrt{d_h}\right), \tag{2}$$

$$\text{MSA}(X) = \text{Concat}(A_1V_1, \ldots, A_HV_H)W^O + b^O. \tag{3}$$

Softmax chạy theo chiều token key; hệ số $\sqrt{d_h}$ kiểm soát độ lớn tích vô hướng. Với $d = 64, H = 4$, mỗi head rộng 16. Trọng số QKV được gộp trong một Linear để thuận tiện đồng bộ với `nn.MultiheadAttention`.

```
qkv = self.qkv(inputs).reshape(B, N, 3, H, d_h)
q, k, v = qkv.permute(2, 0, 3, 1, 4)
scores = (q @ k.transpose(-2, -1)) / math.sqrt(d_h)
weights = self.dropout(scores.softmax(-1))
context = (weights @ v).transpose(1, 2).reshape(B, N, d)
return self.out(context)
```

Đây là phần tính toán MSA tự viết trong `ScratchMSA`; `TorchMSA` gọi `nn.MultiheadAttention` với `batch_first=True` với $Q = K = V = X$, `need_weights=False`. Bản tự viết không thay thế toàn bộ autograd: phép đạo hàm vẫn do PyTorch cung cấp.

5.2 Encoder phân loại ảnh

Mỗi tokenizer tạo token rộng 64. Cộng positional embedding học được, qua hai block pre-LayerNorm, mỗi block gồm residual attention và FFN $64 \to 128 \to 64$ với GELU. Sau LayerNorm cuối, lấy trung bình token rồi Linear $64 \to 10$. Pipeline này *không* dùng class token. Attention học quan hệ token toàn cục; chất lượng còn tùy tokenization và lượng dữ liệu, không tự động hơn CNN.

5.3 Kiểm chứng tương đương với PyTorch

Để so sánh toán tử thay vì hai khởi tạo ngẫu nhiên khác nhau, notebook tạo reference scratch rồi chép toàn bộ trọng số tương ứng sang target PyTorch. Tầng tokenizer, position, normalization, FFN và classifier cũng được đồng bộ trong cặp cấu hình.

```
torch_msa.mha.in_proj_weight.data.copy_(scratch.qkv.weight)
torch_msa.mha.in_proj_bias.data.copy_(scratch.qkv.bias)
torch_msa.mha.out_proj.weight.data.copy_(scratch.out.weight)
torch_msa.mha.out_proj.bias.data.copy_(scratch.out.bias)

x1 = x.clone().requires_grad_(True)
x2 = x.clone().requires_grad_(True)
y1, y2 = scratch(x1), torch_msa(x2)
# Compare y1/y2, then backward the same scalar objective.
# Compare x1.grad/x2.grad and corresponding parameter grads.
```

Các phép so sánh chạy CPU FP32, dropout 0, ngưỡng `atol=1e-5, rtol=1e-4`. Kiểm tra output chứng minh forward đúng trên input probe; kiểm tra input gradient xác nhận đường lan truyền về tầng trước; kiểm tra từng parameter gradient kiểm tra cập nhật các tham số tương ứng. Chỉ kiểm tra forward là chưa đủ để chứng minh bản tự viết sẵn sàng train.

Output đã lưu của cell kiểm chứng E2:

```
MSA output, input gradients and parameter gradients match
Tokenizer shapes are valid
E2 one-batch overfit: 2.4051413536071777 -> 0.04063470661640167
```

Log không in maximum absolute/relative error; do đó báo cáo chỉ ghi phép assert đã qua, không gán sai số số học cụ thể. One-batch overfit kiểm tra plumbing model/loss/optimizer có khả năng giảm loss trên một batch, không phải kết quả generalization.

Mặc dù khởi tạo được ghép cặp, các kernel số học và đường thực thi train có thể khác. Chênh lệch accuracy giữa scratch/PyTorch sau 20 epoch không phủ nhận kiểm chứng toán tử ở ngưỡng đã chọn; cũng không chứng minh một implementation có lợi thế tổng quát. Cần nhiều seed để đánh giá tính ổn định.

Nguồn: `E2/e2_attention_tokenization.ipynb`.

5.4 Ba tokenizer và sáu cấu hình

Bảng 5: Tokenization E2. Mọi output đều có chiều embedding 64.

Tokenizer	N	Input/token	Cơ chế
Patch	49	$4 \times 4 = 16$	Unfold không chồng lấp rồi Linear $16 \to 64$
Row	28	28 pixel	Mỗi hàng là một token; Linear $28 \to 64$
CNN-stem	49	Đặc trưng CNN	Hai Conv 3 stride 2 padding 1, kênh $1 \to 32 \to 64$; flatten 7×7

Patch giữ nhóm lân cận hai chiều nhưng phải học biểu diễn từ pixel thô. Row giảm số token, giữ chiều ngang trong mỗi vector nhưng phụ thuộc thứ tự hàng; độ phức tạp ma trận attention giảm từ 49^2 xuống 28^2. CNN-stem đưa bias cục bộ vào tokenizer, thêm tham số và tính toán convolution trước attention. Do đó so patch và CNN-stem là so cả biểu diễn lẫn capacity, không phải một ablation chỉ đổi số token.

Bảng 6: E2: kết quả đo; best epoch đổi từ chỉ số 0 sang thứ tự 1–20.

ID	Val (%)	Test (%)	Test loss	Best*	Tham số
scratch_patch	89.13	88.09	0.347	17	71,946
scratch_row	90.52	89.23	0.327	17	71,370
scratch_cnn	91.50	90.20	0.307	18	89,674
torch_patch	89.50	88.04	0.330	18	71,946
torch_row	90.33	88.34	0.341	14	71,370
torch_cnn	91.20	89.75	0.295	15	89,674

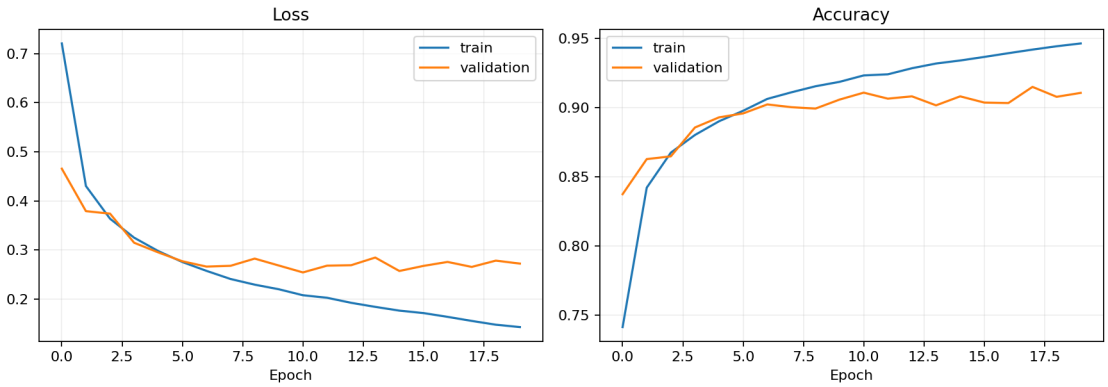
Theo validation, scratch CNN-stem đứng đầu sáu cấu hình; test 90,20% cũng cao hơn patch và row tương ứng. Cùng tokenizer, sai khác scratch/PyTorch tương đối nhỏ so với khác biệt biểu diễn, nhưng một seed chưa đủ kết luận implementation tương đương về phân phối kết quả train. Row có footprint attention nhỏ hơn; song latency phụ thuộc backend và kernel, nên không dùng

Bảng 7: E2: chi phí đo trên Tesla T4. Ký hiệu MB trong JSON được tính theo 2^{20} byte.

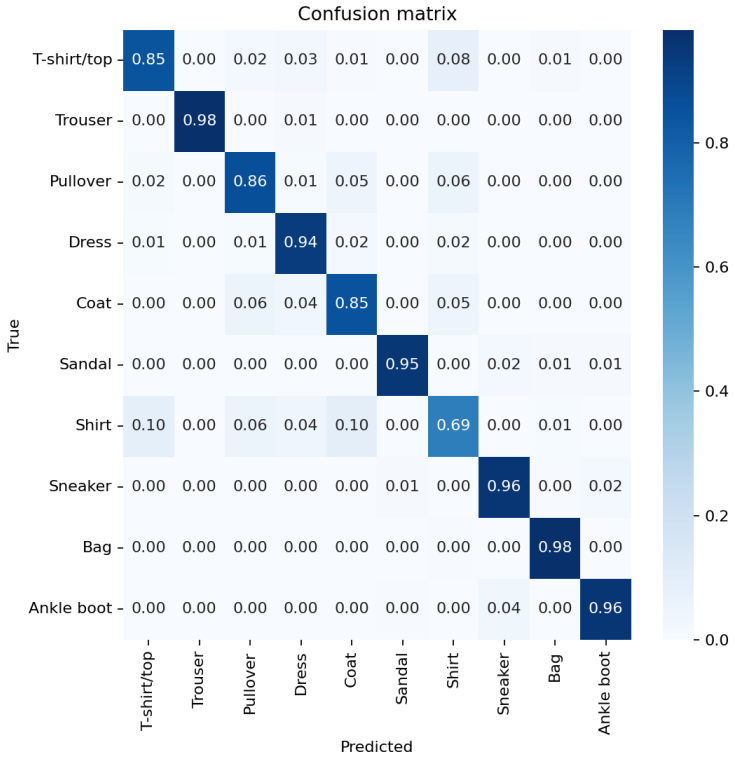
ID	VRAM train (MiB)	Weights (MiB)	ms/ảnh (B32)
scratch_patch	64.34	0.274	0.0429
scratch_row	41.27	0.272	0.0359
scratch_cnn	68.36	0.342	0.0490
torch_patch	51.74	0.274	0.0343
torch_row	41.57	0.272	0.0272
torch_cnn	54.19	0.342	0.0381

riêng công thức $O(N^2 d)$ để dự báo thời gian tuyệt đối.

5.5 Động học học và lỗi E2



Hình 5: Learning curves cấu hình E2 scratch CNN-stem; ảnh sao chép từ artifact, không tái tạo CSV thiếu.



Hình 6: Nhầm lẫn E2 scratch CNN-stem trên 10 lớp. Xem cùng nhãn trục khi đối chiếu CNN E1.

Kết hợp attention với CNN tokenizer chưa vượt CNN E1 trong study này. Giải thích hợp lý là bias cục bộ có ích cho ảnh 28 pixel, trong khi encoder attention tăng bài toán tối ưu và capacity. Đây là diễn giải từ thiết kế và kết quả, không phải thử nghiệm nhân quả tách riêng từng thành phần. Các lỗi nhóm áo vẫn cần xem cùng ma trận E1 để tránh chỉ dựa trên accuracy tổng.

6 E3: LSTM và GRU trên chuỗi ảnh

6.1 Ánh xạ ảnh thành chuỗi

Row dùng $X \in \mathbb{R}^{B\times 28\times 28}$: thời gian chạy từ hàng trên xuống dưới. Patch dùng `unfold` với kernel4, stride4, tạo $B \times 49 \times 16$ theo thứ tự quét các patch. Cả hai không dùng positional embedding riêng; thứ tự được đưa vào thông qua recurrence.

```
if representation == "row":
    sequence = images.squeeze(1)  # B, 28, 28
else:
    sequence = F.unfold(images, 4, stride=4).transpose(1, 2)
output, state = self.recurrent(sequence)
hidden = state[0] if isinstance(state, tuple) else state
return self.head(hidden[-1])
```

Mỗi cấu hình dùng một tầng hồi tiếp một chiều, hidden 128 và Linear $128 \to 10$. Với LSTM, state là (h_n, c_n) nên lấy thành phần h_n; GRU trả trực tiếp h_n. Classifier nhận trạng thái ẩn cuối, không nhận toàn bộ các output theo thời gian.

6.2 Cơ chế cổng

LSTM [2] có cell state để truyền thông tin qua chuỗi:

$$i_t, f_t, o_t = \sigma(W_{i,f,o}[x_t; h_{t-1}] + b_{i,f,o}), \tag{4}$$

$$\widetilde{c}_t = \tanh(W_c[x_t; h_{t-1}] + b_c), \tag{5}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \widetilde{c}_t, \quad h_t = o_t \odot \tanh(c_t). \tag{6}$$

GRU [1] dùng update/reset gate, không tách cell state. Có thể viết khái quát $h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$, trong đó candidate n_t phụ thuộc reset gate. Khi đối chiếu implementation, cần chú ý PyTorch áp dụng reset gate lên phép biến đổi hidden trong candidate; không coi mọi quy ước viết công thức GRU là cùng một phép toán chi tiết.

Gradient clipping 1 giúp hạn chế gradient quá lớn, không bảo đảm chuỗi luôn giữ đủ thông tin hai chiều của ảnh. Các model này học tuần tự theo một trật tự nhân tạo trên ảnh tĩnh.

6.3 Kết quả E3 và đối chiếu baseline

Bảng 8: E3: kết quả đo; best epoch đổi từ chỉ số 0 sang thứ tự 1–20.

ID	Val (%)	Test (%)	Test loss	Best*	Tham số
lstm_row	90.82	89.75	0.292	17	82,186
lstm_patch	89.27	88.40	0.334	20	76,042
gru_row	91.25	89.87	0.296	14	61,962
gru_patch	89.78	88.25	0.335	16	57,354

Bảng 9: E3: chi phí đo trên Tesla T4. Ký hiệu MB trong JSON được tính theo 2^{20} byte.

ID	VRAM train (MiB)	Weights (MiB)	ms/ảnh (B32)
lstm_row	40.04	0.314	0.0157
lstm_patch	48.54	0.290	0.0284
gru_row	39.77	0.236	0.0103
gru_patch	49.11	0.219	0.0201

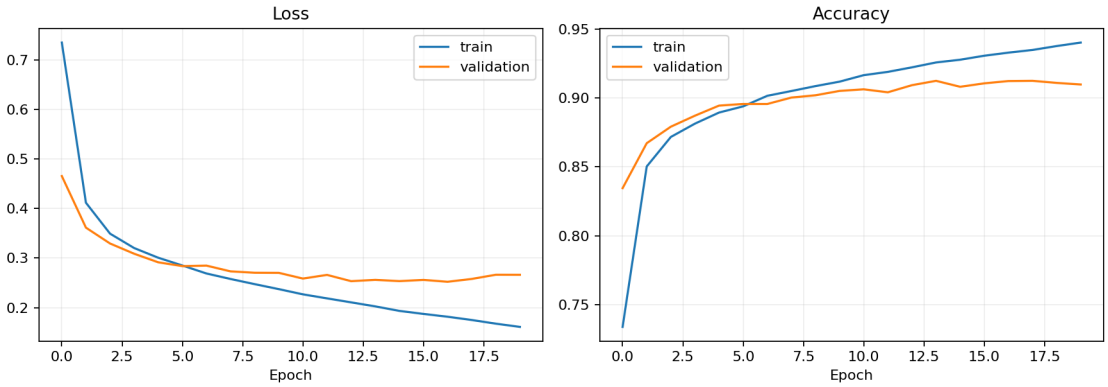
Row tốt hơn patch ở cả LSTM và GRU về test accuracy, lần lượt 1,35 và 1,62 điểm phần trăm. Patch làm chuỗi dài 49 thay vì 28, dù vector từng bước nhỏ hơn. Bộ nhớ hidden cuối phải tổng hợp qua nhiều bước hơn, đồng thời patch bị sắp thành một dãy tuyến tính; kết quả chưa cho thấy lợi ích của biểu diễn này ở ngân sách hiện tại.

GRU-row đạt 89,87%, cao hơn LSTM-row 0,12 điểm nhưng chênh lệch này nhỏ và chưa có nhiều seed. Nếu chọn chỉ theo validation, GRU-row cũng đứng đầu bốn cấu hình. CNN E1 vẫn cao hơn GRU-row 2,08 điểm test; MLP E1 thấp hơn GRU-row 0,54 điểm. Các baseline được tái sử dụng từ E1 sau khi xác nhận cùng seed/split hash, không huấn luyện lại để tạo một đối chứng mới.

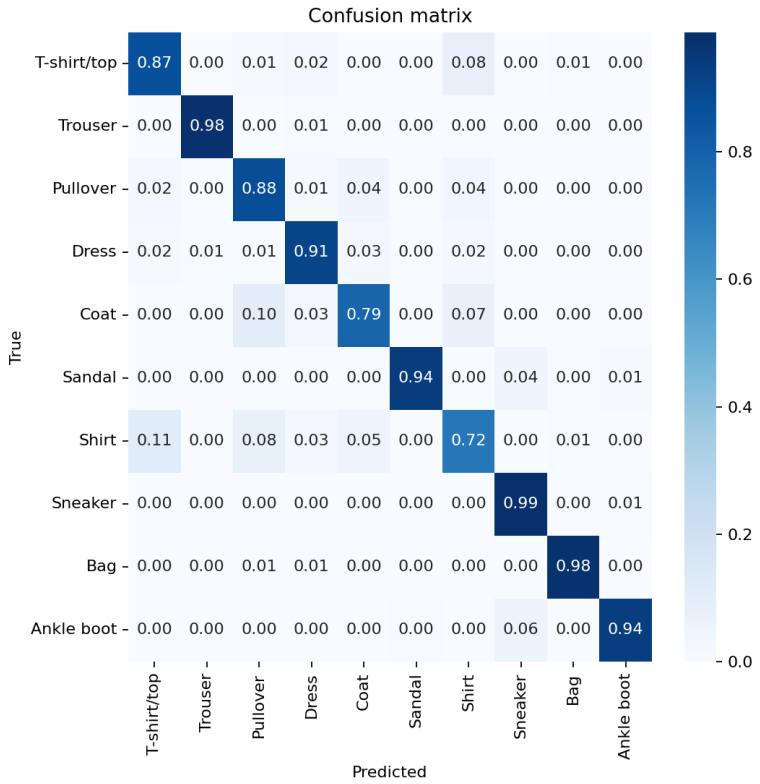
GRU-row dùng 61.962 tham số so với 82.186 của LSTM-row. Kết hợp accuracy và latency đo được khiến GRU-row là phương án RNN phù hợp hơn trong study này. CNN E1 vẫn có accuracy cao nhất trong 13 cấu hình, nhưng lựa chọn thực tế còn phụ thuộc hạn chế latency/footprint của ứng dụng.

Nguồn: `E3/e3_sequence_models.ipynb`.

6.4 Learning curves và lỗi E3



Hình 7: Learning curves GRU-row; best checkpoint ở epoch 14 theo thứ tự 1–20.



Hình 8: Ma trận nhầm lẫn GRU-row từ export E3.

Đường cong cho thấy cần đọc validation cùng train khi đánh giá generalization; loss train giảm không tự suy ra loss test giảm. Tập lớp quần áo gần nhau về đường viền vẫn là thử thách, dù dùng gating thay cho convolution hoặc attention. Chưa có failure grid riêng E3 trong bundle, vì vậy báo cáo không gán hình E1 thành lỗi của E3.

7 Tổng hợp, hạn chế và hướng phát triển

7.1 Trade-off quan sát được

Ba bài cho thấy biểu diễn và inductive bias quan trọng bên cạnh số tham số. MLP lớn hơn CNN nhưng accuracy thấp hơn. Tokenizer CNN-stem giúp E2 tốt hơn patch thô trong các run hiện có. Row giúp E3 rút ngắn chuỗi và đạt chất lượng cao hơn patch. Các quan sát nhất quán với việc khai thác cấu trúc ảnh, song không chứng minh kiến trúc riêng lẻ là nguyên nhân duy nhất: optimizer, capacity, token length và kernel cũng thay đổi.

Về chi phí, số tham số quyết định phần lớn dung lượng weights nhưng không mô tả đủ activation hoặc số phép toán. Latency B 32 có thể tốt với dense layer ngay cả khi nhiều tham số hơn convolution. Không diễn giải số ms/ảnh B 32 như độ trễ phục vụ tương tác một ảnh. Khi deploy cần đo lại đúng batch, precision và thiết bị đích.

7.2 Giới hạn bằng chứng

Chỉ một seed và 20 epoch: chưa ước lượng phương sai giữa các run, chưa thử tối ưu siêu tham số ngang mức công sức, chưa chứng minh hội tụ. Chuẩn hóa dùng official train trước split là một hạn chế cần công bố. Các notebook tương ứng có kiểm thử shape, one-batch overfit và kiểm chứng MSA; chúng giúp phát hiện lỗi implementation nhưng không thay thế đánh giá test.

Macro-F1, calibration và các khoảng tin cậy không được ghi trong metrics E1–E3. Không thể suy ra chúng chính xác chỉ từ accuracy hoặc PNG đã làm tròn. E2 không xuất history CSV trong bộ artifact hiện có; hình đường cong là bằng chứng ở mức hình ảnh. Báo cáo giữ các khoảng trống này thay vì điền số đo ước lượng.

7.3 Hướng cải thiện có thể kiểm chứng

Một study tiếp theo nên khóa protocol train-only normalization, lặp nhiều seed, cố định ngân sách compute và khảo sát augmentation nhẹ. Với E2, có thể giữ số tham số/token gần nhau khi ablate tokenizer để giảm nhiễu capacity. Với E3, thử pooling toàn chuỗi hoặc hai chiều, nhưng phải báo riêng chi phí và không nhập chung vào bốn cấu hình hiện tại. Việc hiệu chỉnh xác suất cần validation riêng; test chỉ dùng một lần sau khi quyết định.

8 Tái lập và cấu trúc tài liệu

8.1 Notebook, artifact và báo cáo

Ba notebook độc lập chứa dataset, model, training và evaluation. Nguồn trong repository:

- `E1/e1_fashion_mnist_baselines.ipynb`
- `E2/e2_attention_tokenization.ipynb`
- `E3/e3_sequence_models.ipynb`

Bảng báo cáo sinh bằng `pythontools/prepare_reports.py`. Công cụ đọc các file metrics, config và environment, kiểm tra identity và lưu SHA-256 đầu vào trong `reports/latex/generated/provenance.json`. Không nạp checkpoint pickle, không import torch và không chạy training. Hình được sao chép từ artifact hoặc output EDA notebook có ghi hash nguồn.

Bản LaTeX dùng template HCMUT dạng article/vntex; style được tách vào `reports/latex/shared/`. Build ba báo cáo bằng `pythontools/build_reports.py`. Muốn tái chạy thí nghiệm cần môi trường ML và dữ liệu; build báo cáo chỉ cần artifact nhẹ và TeX Live, không cần GPU.

8.2 Định danh và cách kiểm toán

Mỗi hàng bảng truy ngược được bằng bài và experiment ID. Ví dụ `E2/scratch_cnn` tương ứng `artifacts/E2/full/scratch_cnn/metrics.json`. Các bảng chi phí cùng ID đọc đúng metrics tương ứng, không ghép hình hay số đo của study khác. Epoch hiển thị được cộng 1 nhất quán; số liệu gốc vẫn giữ trong provenance để đối chiếu.

Các PDF là báo cáo thực nghiệm được biên soạn từ bằng chứng đã có, không chứng minh đã hoàn tất mọi thủ tục nộp bài. URL video và xác nhận các mốc nộp bài vẫn là phần các tác giả cần bổ sung. Bản báo cáo cùng nguồn có thể đưa lên Overleaf bằng pdfLaTeX; thư mục shared và generated phải đi kèm thư mục từng bài.

Tài liệu

[1] Kyunghyun Cho et al. Learning phrase representations using rnn encoder–decoder for statistical machine translation. https://arxiv.org/abs/1406.1078, 2014.

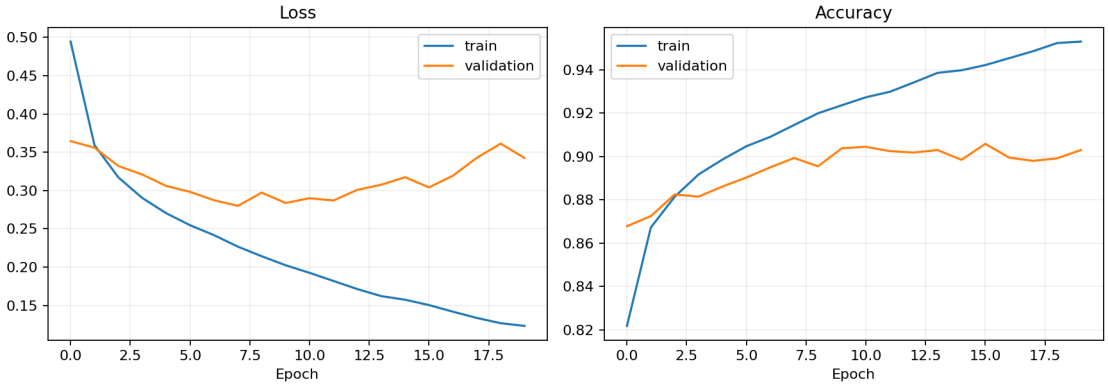
[2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[3] Ashish Vaswani et al. Attention is all you need. In *NeurIPS*, 2017. https://arxiv.org/abs/1706.03762.

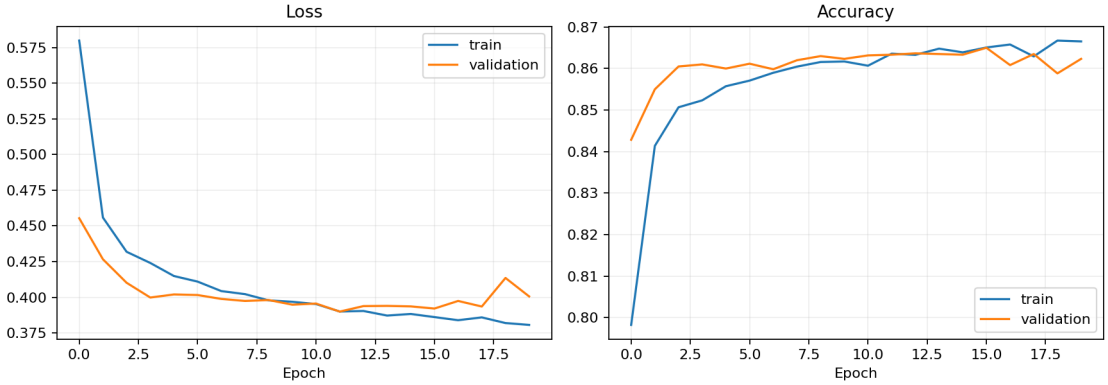
[4] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. https://arxiv.org/abs/1708.07747, 2017.

[5] Zalando Research. Fashion-mnist: official dataset repository. https://github.com/zalandoresearch/fashion-mnist, 2017.

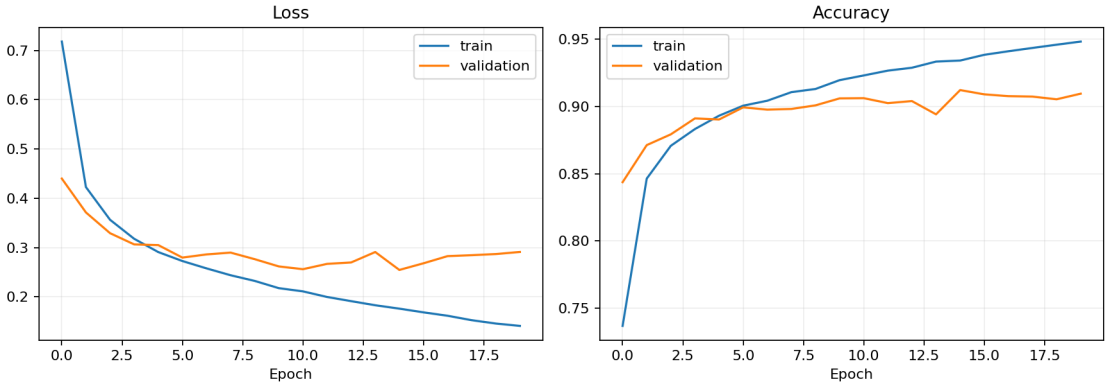
A Đường cong và ma trận bổ sung



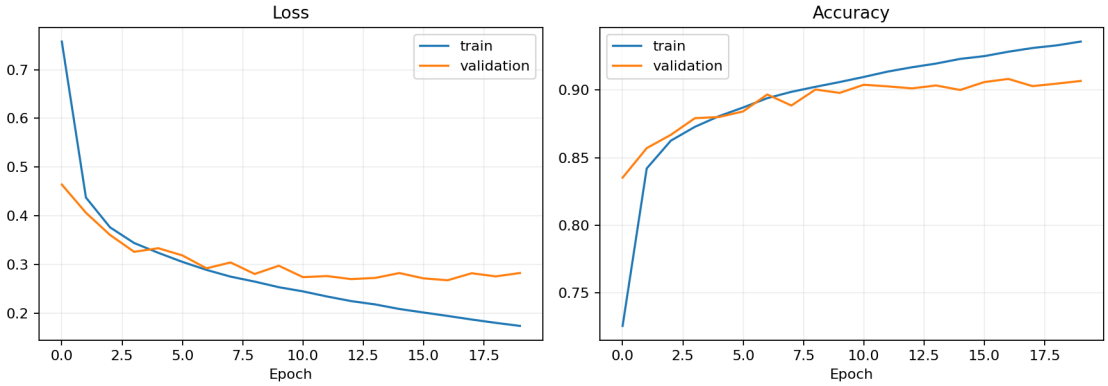
Hình 9: MLP E1: đường cong đối chứng với CNN và softmax.



Hình 10: Softmax E1: baseline tuyến tính với cùng ngân sách.



Hình 11: E2 PyTorch attention và CNN-stem; cùng thiết kế encoder với bản tự viết.



Hình 12: E3 LSTM-row để đối chiếu GRU-row.